Agente inteligente con RAG sobre la API REST de GitHub

Informe técnico · Trabajo de Laboratorio N° 2 · Inteligencia Artificial – Sistemas Inteligentes · FCEFyN – UNC

Integrantes: _____ Fecha: octubre de 2026

1. Corpus técnico: construcción, limpieza y segmentación

Se eligió como dominio la **API REST de GitHub** (versión 2022-11-28): es una especificación formal y acotada en la que los LLMs suelen inventar rutas, parámetros y códigos de estado, por lo que el RAG aporta precisión verificable. El corpus son 4 documentos Markdown (≈21.000 caracteres) redactados a partir de la documentación oficial (docs.github.com/rest): autenticación y límites de uso; repositorios, ramas y releases; issues y etiquetas; pull requests y revisiones. Cada endpoint tiene su propio encabezado con método, ruta, parámetros y un ejemplo JSON. Se verificaron los datos contra la documentación oficial y se corrigieron imprecisiones (por ejemplo, sólo `User-Agent` es obligatorio y el exceso de cuota devuelve 403 o 429).

- **Limpieza:** normalización de saltos de línea y eliminación de separadores `---`, marcas de negrita y líneas vacías repetidas. Se conservan tablas y bloques de código porque contienen los datos exactos (rutas, campos JSON). Se agregó el sinónimo "campos obligatorios" junto a "parámetros requeridos", que es como los usuarios formulan las preguntas.
- **Segmentación estructural:** `MarkdownHeaderTextSplitter` corta por encabezados `#`/`##`/`###`, de modo que cada chunk es una sección completa, es decir, un endpoint con su tabla o lista de parámetros. `RecursiveCharacterTextSplitter` (1500 caracteres, solapamiento 150) sólo actúa como resguardo si una sección supera ese tamaño. Una primera versión usaba 800 caracteres y partía tablas entre filas: el agente veía sólo la fila de `User-Agent` y omitía el resto de las cabeceras. A cada chunk se le antepone su ruta de encabezados (p. ej. `[Issues y Etiquetas > 1.2 Crear un Issue]`). Resultado: 39 chunks de 562 caracteres en promedio (máx. 1429, ≈ 400 tokens, dentro del límite de 512 del modelo de embeddings).

2. Base vectorial y técnica de búsqueda

Embeddings: `intfloat/multilingual-e5-small` (384 dimensiones), elegido porque corpus y preguntas están en español; usa los prefijos `query:`/`passage:` con los que fue entrenado. **Base vectorial:** FAISS con `IndexFlatIP`; como los vectores están normalizados, el producto interno es la similitud coseno y la búsqueda es exacta (con 39 vectores no hace falta un índice aproximado). **Técnica de búsqueda: híbrida.** Un modelo de embeddings pequeño puede pasar por alto términos exactos (`title`, `422`, "obligatorio"), así que se combina con **BM25** (ranking léxico implementado sobre los mismos chunks). Cada buscador aporta 10 candidatos y se fusionan con **Reciprocal Rank Fusion**, RRF(d) = Σ 1/(60 + rango), que usa sólo posiciones y no requiere calibrar puntajes. El LLM recibe los 4 mejores. Evaluación con 16 preguntas parafraseadas de sección conocida (k = 4):

Método de búsqueda (mismos chunks)	Hit@1	Hit@4	MRR
Semántica · all-MiniLM-L6-v2 (entrenado en inglés)	0,38	0,62	0,48
Semántica · multilingual-e5-small	0,69	0,94	0,78
Léxica · BM25	0,75	0,94	0,83
Híbrida · e5 + BM25 con RRF (elegida)	0,88	0,94	0,89

La búsqueda híbrida es la mejor en las tres métricas: ubica primero la sección correcta en el 88 % de las preguntas, frente al 69 % de e5 y al 75 % de BM25, porque cada buscador compensa los fallos del otro. El modelo entrenado en inglés queda muy por debajo (38 %), lo que confirma la elección de un modelo multilingüe.

3. Agente: herramientas y flujo de trabajo

LLM: `qwen2.5:7b` ejecutado localmente con **Ollama** (sin claves de API; temperatura 0 y contexto ampliado a 8192 tokens para que entren los fragmentos). **Orquestación:** `create_agent` de LangChain, que implementa el ciclo ReAct mediante *tool calling*. El agente tiene tres herramientas; el LLM elige cuáles usar a partir de su nombre y descripción:

Herramienta	Tipo	Qué hace
`consultar_documentacion_github(consulta)`	RAG	Búsqueda híbrida en el corpus; devuelve los 4 fragmentos más relevantes con su archivo de origen.
`consultar_repositorio_github(owner, repo)`	Acción externa en vivo	Llama a la API real: `GET /repos/{owner}/{repo}` y `GET /search/issues` para separar issues de pull requests abiertos (el campo `open_issues_count` los suma).
`calcular_cuota_rate_limit(solicitudes_por_hora, autenticado)`	Cálculo	Límite aplicable (60 o 5000 por hora), porcentaje consumido, solicitudes restantes o excedentes e intervalo mínimo entre solicitudes.

Flujo paso a paso (ciclo ReAct):

1. El usuario envía la consulta. El LLM la recibe junto con las instrucciones del sistema y la descripción de cada herramienta.
2. El LLM identifica las partes de la pregunta y emite una o varias *tool calls* (nombre + argumentos JSON). No ejecuta nada por sí mismo.
3. LangChain ejecuta las funciones de Python: búsqueda híbrida en el corpus, petición HTTP a GitHub y/o cálculo de cuota.

4. Los resultados vuelven al LLM como observaciones; si necesita más datos repite el ciclo (máximo 12 pasos).
5. El LLM redacta la respuesta final en español usando sólo esas observaciones: cita el archivo fuente ([Fuente: ...]), aclara qué datos son en vivo y, si la documentación no cubre el tema, lo dice en lugar de inventar.

4. Ejemplo real de interacción

Ejecución real del notebook (17/10/2026) con `qwen2.5:7b` en CPU (Ryzen 5 5600G, 16 GB de RAM, sin GPU). Las respuestas están resumidas; las completas, con la traza de acciones y observaciones, están en la sección 10 del notebook. La pregunta 5 está fuera del corpus a propósito, para verificar que el agente no invente.

Pregunta del usuario	**Herramientas invocadas**	**Respuesta del agente (resumida)**
1. ¿Qué cabeceras HTTP se recomienda enviar en cada solicitud a la API REST de GitHub y qué ocurre si una petición no incluye el User-Agent?	Documentación (RAG)	Enumera `Accept: application/vnd.github+json`, `Authorization: Bearer <TOKEN>`, `X-GitHub-Api-Version: 2022-11-28` y `User-Agent` (obligatorio por política de GitHub). Sin `User-Agent` la petición se rechaza con HTTP 403 [Fuente: 01_autenticacion_y_rate_limits.md]. (86 s)
2. Mi script hace 80 solicitudes por hora sin token. ¿Supera el límite? ¿Y con un token personal? ¿Qué código de error devolvería GitHub?	Cálculo (×2) + documentación	Sin token supera el límite de 60/h (133,3 % de la cuota, 20 solicitudes de exceso); con token queda dentro del límite de 5000/h (1,6 %, quedan 4920). Al excederlo GitHub responde `429 Too Many Requests`, con `x-ratelimit-remaining` en 0 hasta el instante de `x-ratelimit-reset` [Fuente: 01_autenticacion_y_rate_limits.md]. (82 s)
3. ¿Cuántas estrellas, forks, issues abiertos y pull requests abiertos tiene actualmente facebook/react, y cuál es su lenguaje principal?	Repositorio en vivo	251.223 estrellas, 51.516 forks, 880 issues abiertos y 579 pull requests abiertos; lenguaje principal JavaScript (datos obtenidos en vivo de la API de GitHub). (23 s)
4. ¿Qué método HTTP y endpoint se usan para crear un issue, qué campo es obligatorio y cuántos issues abiertos tiene hoy facebook/react?	Documentación + repositorio en vivo	`POST /repos/{owner}/{repo}/issues`; el campo obligatorio es `title` [Fuente: 03_issues_y_etiquetas.md]. El repositorio tiene actualmente 880 issues abiertos (datos en vivo). (56 s)
5. Según la documentación, ¿qué endpoint se usa para crear un webhook en un repositorio?	Documentación (RAG)	La documentación del corpus no contiene información sobre un endpoint para crear un webhook en un repositorio. (33 s)

5. Conclusiones

El pipeline completo (corpus → búsqueda híbrida → agente con tres herramientas) se ejecuta sin errores con un LLM local y responde correctamente las cinco consultas de prueba, incluida la que está fuera del corpus. Tres decisiones surgieron de analizar errores reales de versiones anteriores. Los chunks pasaron a ser de sección completa, porque con 800 caracteres se partía la tabla de cabeceras. La búsqueda pasó a ser híbrida, porque la sólo semántica no recuperaba "Crear un Issue" y el agente afirmó que `body` era obligatorio. Y se agregaron instrucciones explícitas contra la alucinación, porque el agente inventaba un endpoint de webhooks. **Limitaciones:** latencia de 20 a 90 s por consulta en CPU; corpus acotado a cuatro áreas de la API; respuestas a veces parciales del modelo de 7B (en la pregunta 2 menciona sólo el código 429, aunque la documentación indica 403 o 429).